

Перечень заданий курса Школа Frontend

Задание 1

Задание по теме «Основы HTML»

Описание

Необходимо создать простейшую web-страницу, используя HTML-теги для создания заголовков и списков.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/index.html` внутри `<body>` сделайте страницу следующего вида:

- Заголовок первого уровня
- Заголовок второго уровня
- Нумерованный список из трёх элементов, каждый из которых ссылка на какой-то ресурс

Проверить себя можно запустив команду `npm run test -- --watchAll`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать с базовой структурой HTML-документа.
- Умение использовать заголовки `<h1>` и `<h2>`.
- Умение создавать нумерованные списки.
- Умение добавлять ссылки на внешние ресурсы.
- Умение оформлять результат работы через pull request.

Ссылка на репозиторий

<https://github.com/tensor-school/task-html>

Задание 2

Задание по теме «Флексы»

Описание

Необходимо оформить существующую HTML-разметку с помощью CSS и Flexbox. Использовать подготовленный и подключенный файл для стилей, не изменяя сам HTML-файл. Задать нужные размеры и цвета элементам. А так же задать элементам верное расположение.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/App.css` опишите стили для вёрстки из файла `index.html`

1. Файл `src/index.html` редактировать нельзя
2. Контейнер должен быть шириной `700px` и высотой `300px`
3. Элементы - круги `100x100px` (почитайте про свойство `border-radius`).
4. Цвет элементов - `#DAA520`
5. Первый и пятый элементы должны быть прижаты к верхнему краю оси `Y`
6. Второй и четвертый должны располагаться посередине по оси `Y`
7. Третий должен находится внизу по оси `Y`
8. Сами элементы должны располагаться по всей оси `X` контейнера с одинаковыми промежутками между ними

Итоговый вариант будет выглядеть вот так:

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать с CSS.
- Умение использовать Flexbox для расположения элементов.
- Умение задавать размеры контейнера и элементов.
- Умение создавать круглые элементы с помощью свойства `border-radius`.
- Умение управлять расположением отдельных элементов по поперечной оси.
- Умение равномерно распределять элементы по основной оси контейнера.

Ссылка на репозиторий

<https://github.com/tensor-school/task-flexbox>

Задание 3

Задание по теме «Способы подключения скрипта»

Описание

Необходимо подключить к странице встроенный и несколько внешних JavaScript-скриптов.

Требуется подключить обычный внешний скрипт, скрипт с атрибутом `async` и скрипт с атрибутом `defer`.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/index.html` добавьте на страницу следующие скрипты:

- Встроенный скрипт, который будет исполнять следующий код:
- Внешний скрипт `src/scripts/sync.js`
- Внешний скрипт `src/scripts/async.js` с атрибутом `async`
- Внешний скрипт `src/scripts/defer.js` с атрибутом `defer`

Проверить себя можно запустив команду `npm run test -- --watchAll`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение подключать JavaScript к HTML-документу.
- Умение создавать встроенные скрипты с помощью тега `<script>`.
- Умение подключать внешние JavaScript-файлы.
- Понимание работы обычного синхронного подключения скриптов.
- Понимание назначения атрибута `async`.
- Понимание назначения атрибута `defer`.
- Умение правильно использовать атрибут `src` для подключения внешних скриптов.

Ссылка на репозиторий

<https://github.com/tensor-school/task-scripts>

Задание 4

Задание по теме «Базовый синтаксис JavaScript»

Описание

Необходимо написать код, который преобразует число из римской записи в арабскую. На вход функция получает строку, а результатом её работы должно быть число.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/BasicSyntax.js` в функции `romanToInteger` напишите код, преобразующий число из римской записи в арабскую. На вход вы получите строку, ответом должно быть число.

Символ	Значение
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать со строками.
- Умение использовать условия и циклы.
- Умение реализовывать алгоритм.
- Умение возвращать результат из функции.
- Умение работать с существующей функцией в заданном файле.

Ссылка на репозиторий

<https://github.com/tensor-school/task-basic-syntax>

Задание 5

Задание по теме «Циклы»

Описание

Необходимо реализовать три функции. В каждой функции требуется использовать определённый вид цикла: `for`, `while` или `do while`.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/Cycles.js` создайте функции `rangeSum()`, `iterationCount()` и `symbolsReplace()`, реализующие следующий функционал:

В функцию `rangeSum()` приходят два целых неотрицательных числа. Используя цикл `for`, просуммируйте все четные числа в диапазоне между этими значениями (включительно) и верните итоговый результат.

В функцию `iterationCount()` приходит неотрицательное число. Используя цикл `while`, выполняйте деление этого числа на два до тех пор, пока результат деления больше `0.1` и верните количество потребовавшихся итераций (т.е. сколько раз пришлось выполнить деление).

В функцию `symbolsReplace()` приходит строка текста. Используя цикл `do while`, замените в тексте каждый третий символ на символ нижнего подчеркивания и верните итоговый результат.

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение использовать цикл `for`.
- Умение использовать цикл `while`.
- Умение использовать цикл `do while`.
- Умение работать с диапазоном чисел и находить чётные значения.

- Умение выполнять повторяющиеся математические операции и подсчитывать количество итераций.
- Умение работать со строками и изменять символы по заданному условию.
- Умение возвращать результат выполнения функции.
- Понимание различий между основными видами циклов в JavaScript.

Ссылка на репозиторий

<https://github.com/tensor-school/task-cycles>

Задание 6

Задание по теме «Работа с массивами»

Описание

Необходимо реализовать функции для работы с массивами: сортировки чисел по возрастанию, фильтрации значений и объединения двух массивов с чередованием элементов.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/Arrays.js` Создайте функции `arraySorting()`, `arrayFiltering()` и `arrayPushing()`, реализующие следующий функционал:

1. В функцию `arraySorting()` приходит строка чисел с разделительными запятыми вида `"1,2,33,0,17"`. Преобразуйте строку в массив чисел, отсортируйте их по возрастанию и верните строку с упорядоченными числами, разделенными запятыми.
2. В функцию `arrayFiltering()` приходит массив целых чисел. Преобразуйте массив таким образом, чтобы все элементы массива, в которых содержатся числа более `100`, были бы удалены из массива, и верните его в качестве результата функции.
3. В функцию `arrayPushing()` приходят два массива целых чисел размерностью `5` элементов. Получите третий массив размерностью `10` элементов, в котором последовательно чередуются значения из первых двух массивов, и верните его в качестве результата функции.

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение создавать и обрабатывать массивы.
- Умение преобразовывать строку в массив чисел.
- Умение сортировать числовые значения по возрастанию.
- Умение фильтровать элементы массива по заданному условию.
- Умение объединять два массива в один.
- Умение формировать новый массив с чередованием элементов из двух исходных массивов.
- Умение преобразовывать массив обратно в строку с заданным разделителем.

Ссылка на репозиторий

<https://github.com/tensor-school/task-arrays>

Задание 7

Задание по теме «Функции»

Описание

Необходимо реализовать функции для работы с callback-функциями, методом `filter()` и замыканиями. Требуется создать фильтр, который отбирает числа больше заданного значения, фильтр, который оставляет строки, содержащие указанную подстроку с учетом регистра, а также функцию, возвращающую функцию для последующего умножения числа.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/Functions.js` выполните следующие задания:

У массивов есть метод `arr.filter(callback)`, который позволяет отбирать элементы из массива. `callback` принимает на вход три параметра:

- `element` – текущий элемент
- `index` – индекс текущего элемента
- `array` – исходный массив
- Результатом `callback` должен быть `boolean`:
- `true` – элемент подходит под условие
- `false` – элемент не подходит под условие

Напишите следующие фильтры:

1. `higherThan(num)` – отбирает все числа больше `num`
2. `hasSubstring(substr)` – отбирает из массива все строки, включающие в себя `substr`. Регистр важен, т.е. строка `ABC` не включает в себя подстроку `bc`.

Напишите функцию `multiply(num)`, которая работает вот так:

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение создавать и использовать функции.
- Понимание принципа работы callback-функций.
- Умение использовать метод массива `filter()`.
- Умение создавать функции, возвращающие другие функции.
- Умение передавать параметры во вложенные функции.
- Умение фильтровать числовые значения по заданному условию.
- Умение работать со строками и искать в них подстроки.
- Понимание влияния регистра символов при работе со строками.
- Понимание принципа работы замыканий.

Ссылка на репозиторий

<https://github.com/tensor-school/task-functions>

Задание 8

Задание по теме «Работа с объектами»

Описание

Необходимо реализовать функции для работы с объектами. Требуется изменять свойства объекта в зависимости от значения, получать общий список полей нескольких объектов в алфавитном порядке, а также создавать массив глубоких копий объекта с уникальным полем.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/Objects.js` создайте функции `personUpdate()`, `objectFieldsList()` и `objectClone()`, реализующие следующий функционал:

1. В функцию `personUpdate()` приходят данные в виде объекта, содержащую некую информацию о человеке. Если этот человек является женщиной (свойство `gender` содержит значение `'female'`), то из этого объекта необходимо удалить свойство `age`, если оно есть. Если этот человек является мужчиной (свойство `gender` содержит значение `'male'`), следует убедиться, что в этом объекте есть свойство `income`. Если его нет, необходимо его добавить и присвоить начальное значение `100000`. Объект после манипуляций следует вернуть в качестве результата работы функции.
2. В функцию `objectFieldsList()` приходят три объекта с различными полями, список которых заранее неизвестен. Верните общий список названий этих полей в алфавитном порядке в виде массива строк.
3. Функция `objectClone()` принимает на вход два параметра: объект и число. Верните в результате работы функции `objectClone()` массив с клонами переданного объекта. Необходимо учесть, что передаваемый объект может иметь вложенные объекты. При этом каждый клон должен дополнительно содержать поле `id` со своим порядковым номером в массиве. Количество клонов указывается во втором параметре функции.

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверям

- Умение создавать и изменять объекты.
- Умение проверять наличие свойств объекта.
- Умение добавлять и удалять свойства объекта.
- Умение работать с данными объекта в зависимости от значений его полей.
- Умение получать список ключей объектов.
- Умение объединять поля нескольких объектов и сортировать их в алфавитном порядке.
- Умение создавать копии объектов.
- Понимание различий между поверхностным и глубоким копированием.
- Умение работать с вложенными объектами.
- Умение формировать массивы объектов.

Ссылка на репозиторий

<https://github.com/tensor-school/task-objects>

Задание 9

Задание по теме «Методы и контекст выполнения»

Описание

Необходимо выполнить задания, связанные с контекстом вызова методов. В объекте требуется реализовать подписку и отписку от события, используя один и тот же обработчик и увеличивая значение поля. Также необходимо сделать так, чтобы один метод объекта передавал все полученные аргументы другому методу, но в обратном порядке.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/MethodContext.js` выполните следующие задания:

В объекте `obj` сделайте следующее:

В методе `subscribe` подпишитесь на событие `click` с помощью `EventEmitter.on(eventName, callback)`. В обработчике нужно увеличивать значение поля `count` на 1

В методе `unsubscribe` отпишитесь от события `click` с помощью `EventEmitter.off(eventName, callback)`. В качестве `callback` нужно передавать тот же самый обработчик, который был передан при подписке.

Есть объект `obj1` с двумя методами:

`first` – принимает на вход неограниченное число параметров

`second` – тоже принимает на вход неограниченное число параметров

Сделайте так, чтобы метод `first` вызывал метод `second` со своими аргументами, но в обратном порядке:

```
obj1.first(1, 2, 3);
```

```
// Внутренний вызов должен быть равносильен obj1.second(3, 2, 1)
```

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать с контекстом методов.
- Умение подписываться на события с помощью `EventEmitter.on()`.
- Умение отписываться от событий с помощью `EventEmitter.off()`.
- Понимание необходимости использовать один и тот же callback при подписке и отписке.
- Умение изменять свойства объекта внутри обработчика события.
- Умение работать с неограниченным количеством аргументов функции.
- Умение передавать аргументы из одного метода в другой.
- Умение изменять порядок аргументов перед вызовом другого метода.
- Понимание особенностей вызова методов объекта и сохранения их контекста.

Ссылка на репозиторий

<https://github.com/tensor-school/task-method-context>

Задание 10

Задание по теме «Работа с DOM»

Описание

Необходимо реализовать функции для работы с DOM-деревом. Одна функция должна добавлять на страницу заданный тег с указанным содержимым нужное количество раз. Другая функция должна создавать дерево вложенных `DIV`-элементов с заданным количеством дочерних узлов и глубиной. Третья функция должна создать дерево глубиной 3 с двумя элементами в каждом узле и заменить все элементы второго уровня, сохранив классы, вложенные элементы и остальную структуру дерева.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/DOM.js` создайте функции `appendToBody()`, `generateTree()` и `replaceNodes()`, реализующие следующий функционал:

В функцию `appendToBody()` передаются 3 параметра: `tag` - имя тега, `content` - содержимое тега и `count` - количество вставок. Необходимо, чтобы функция осуществила вставку на страницу указанный тег с указанным содержимым указанное число раз. Считаем, что всегда передается тег, допускающий вставку текста в качестве своего содержимого (`P`, `DIV`, `I` и пр.).

Функция `generateTree()` принимает два параметра - `childrenCount` и `level`. Создайте дерево вложенных тегов `DIV`. Каждый узел дерева должен содержать `childrenCount` узлов. Глубина дерева задается параметром `level`. Каждый элемент должен иметь класс вида `item_n`, где `n` - глубина вложенности элемента. (Нумерацию ведем с единицы). Сформированное дерево верните в качестве результата работы функции.

Создайте функцию `replaceNodes()`. В ней используйте функцию `generateTree()` для создания дерева тегов `DIV` из предыдущего задания. Создайте дерево с вложенностью 3 и числом элементов в каждом узле 2. Далее замените все узлы второго уровня (т.е. имеющие класс `item_2`) на теги `SECTION`. Остальную структуру дерева сохраните неизменной, включая классы и те элементы, которые находились внутри переписанных тегов. Сформированное дерево верните в качестве результата работы функции.

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать с DOM в JavaScript.

- Умение создавать HTML-элементы программно.
- Умение добавлять созданные элементы на страницу.
- Умение работать с параметрами функций при создании элементов.
- Умение создавать вложенную структуру DOM-элементов.
- Понимание принципов рекурсивного построения дерева элементов.
- Умение назначать CSS-классы элементам в зависимости от уровня вложенности.
- Умение находить элементы по классу.
- Умение заменять DOM-узлы на элементы другого типа.
- Умение сохранять дочерние элементы и структуру DOM при замене узлов.

Ссылка на репозиторий

<https://github.com/tensor-school/task-dom>

Задание 11

Задание по теме «JavaScript и стили»

Описание

Необходимо реализовать набор функций для управления CSS-классами DOM-элементов: проверять наличие класса, добавлять и удалять его, а также переключать состояние класса в зависимости от того, присутствует он у элемента или нет.

Текст задания

Установите зависимости:

```
npm install
```

В файле src/Styles.js создайте несколько функций:

hasClass(elem, elemClass) - позволяет проверить, есть ли у элемента заданный CSS-класс.

Возвращает true/false;

addClass(elem, elemClass) - позволяет добавить к элементу указанный CSS-класс. Ничего не возвращает;

removeClass(elem, elemClass) - позволяет удалить указанный CSS-класс у элемента. Ничего не возвращает

toggleClass(elem, elemClass) - позволяет переключить CSS-класс у элемента, то есть добавить, если его нет и удалить, если он присутствует. Ничего не возвращает.

В функции передаются 2 параметра: elem - DOM-элемент, который проверяется, elemClass - название класса, которое проверяется.

Проверить себя можно запустив команду npm run test.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать с DOM-элементами в JavaScript.
- Умение проверять наличие CSS-класса у элемента.
- Умение добавлять CSS-классы к элементам.
- Умение удалять CSS-классы.
- Умение переключать состояние CSS-класса.
- Понимание принципов программного управления стилями элементов.
- Умение работать с параметрами функций и логическими значениями.

Ссылка на репозиторий

<https://github.com/tensor-school/task-js-styles>

Задание 12

Задание по теме «События»

Описание

Необходимо реализовать функции для создания DOM-элементов и обработки событий.

Текст задания

Установите зависимости:

```
npm install
```

В файле src/Events.js создайте функции, реализующие следующий функционал:

Создайте функцию createButton(). Необходимо, чтобы эта функция осуществила вставку в body тег button с текстом: "Удали меня". При клике по button удалить этот button.

Создайте функцию createArrList(arr), в которую передается 1 параметр: arr - массив строк.

Функция выводит этот массив в виде маркированного списка внутри тега body. При наведении курсора мыши на элемент списка у этого элемента создается атрибут title, в котором записан его текст.

Создайте функцию createLink(), которая сгенерирует следующую разметку и вставит ее в body:

```
<a href="https://tensor.ru/">tensor</a>
```

При первом клике по ссылке в конец ее текста через пробел дописывается ее href. При следующем клике происходит действие по умолчанию (переход по ссылке в текущей вкладке).

Создайте функцию createList(), которая сгенерирует следующую разметку и вставит ее в body:

```
<ul>
```

```
  <li>Пункт</li>
```

```
</ul>
```

```
<button>Добавить пункт</button>
```

При клике по элементу li ему в конец текста добавляется восклицательный знак. При клике по button в конец списка добавляется новый элемент li с текстом: "Пункт". Клик по новому li также добавляет восклицательный знак в конец текста.

Проверить себя можно запустив команду npm run test.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение создавать HTML-элементы средствами JavaScript.
- Умение добавлять элементы в `body` документа.
- Умение работать с событиями клика.
- Умение удалять элементы со страницы при наступлении события.
- Умение создавать маркированные списки на основе массива строк.
- Умение обрабатывать событие наведения курсора мыши на элемент.
- Умение добавлять и изменять атрибуты DOM-элементов.
- Умение создавать ссылки и управлять их поведением при клике.
- Умение отменять действие по умолчанию для ссылки.
- Умение динамически добавлять новые элементы списка.
- Умение использовать обработчики событий для уже созданных и новых элементов.

Ссылка на репозиторий

<https://github.com/tensor-school/task-events>

Задание 13

Задание по теме «Типы данных»

Описание

Необходимо типизировать предложенные функции, указав только типы без реализации функций.

Текст задания

Установите зависимости:

```
npm install
```

Выполните задание:

Типизируйте предложенные функции.

Вам необходимо ТОЛЬКО указать типы, писать реализацию НЕ НУЖНО

В файле src/index.d.ts типизируйте следующие функции:

1. Функцию getPersons, возвращающую массив объектов со следующими свойствами:
 - name - строка

- age - число
- gender - 'male' или 'female'

2. Функцию `personToString`, возвращающую строку и принимающую объект одного из следующих форматов:

I. Пользователь:

- name - строка
- age - число
- gender - 'male' или 'female'

II. Сотрудник

- name - строка
- age - число
- gender - 'male' или 'female'
- company- строка

Проверить себя можно запустив команду `npm run test`. Если тесты завершились и не было выведено ни одного сообщения - все в порядке.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение описывать типы функций в TypeScript.
- Умение типизировать функцию, возвращающую массив объектов.
- Умение описывать объектный тип со свойствами.
- Умение использовать строковые литеральные типы.
- Умение типизировать функцию, принимающую объект в качестве параметра.
- Умение описывать объединение типов для разных форматов объектов.
- Умение типизировать функцию, возвращающую строку.

Ссылка на репозиторий

<https://github.com/tensor-school/task-types>

Задание 14

Задание по теме «Классы»

Описание

Необходимо реализовать класс, который хранит значение счётчика, позволяет задавать начальное значение и изменять его с помощью методов увеличения и уменьшения.

Текст задания

Установите зависимости:

`npm install`

В файле `src/Classes.ts` реализуйте класс `Counter`:

Значение счётчика хранится в поле `counter: number`. Значение поля по умолчанию: 0

В конструктор можно передать параметр `start: number` - начальное значение счётчика.

Класс должен иметь метод `increment`, который увеличивает `counter` на 1 и возвращает результат

Класс должен иметь метод `decrement`, который уменьшает `counter` на 1 и возвращает результат

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение создавать классы в TypeScript.
- Умение задавать значение поля по умолчанию.
- Умение использовать конструктор класса с параметром.
- Умение возвращать результат работы методов.

Ссылка на репозиторий

<https://github.com/tensor-school/task-classes>

Задание 15

Задание по теме «Интерфейсы»

Описание

Необходимо исправить ошибку в данных, возвращаемых функцией, а также реализовать функцию, которая возвращает массив объектов с данными студентов в соответствии с интерфейсом.

Текст задания

Установите зависимости:

```
npm install
```

В файле src/Interfaces.ts выполните следующие задания:

Найдите и исправьте ошибку в данных, возвращаемых функцией `getPerson`.

Допишите функцию `getStudentsData` так, чтобы она возвращала массив из указанных в комментарии данных. Данные должны храниться в объектах, соответствующих интерфейсу `IStudentData`.

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать с интерфейсами в TypeScript.
- Умение находить и исправлять ошибки в данных, возвращаемых функцией.
- Умение создавать объекты, соответствующие заданному интерфейсу.
- Умение работать с массивами объектов.
- Умение реализовывать функции с типом возвращаемого значения.
- Умение корректно заполнять обязательные и необязательные поля объекта.

Ссылка на репозиторий

<https://github.com/tensor-school/task-interfaces>

Задание 16

Задание по теме «Статические поля и методы»

Описание

Необходимо установить зависимости проекта и доработать один из классов: добавить публичное поле экземпляра, публичное статическое поле, а также публичные статические методы для получения значения и округления числа в большую сторону.

Текст задания

Установите зависимости:

```
npm install
```

В файле src/Static.ts выполните следующие задания:

Добавьте в класс `TestClass` публичное поле `counter` с начальным значением 0

Добавьте в класс `TestClass` публичное статическое поле `counterStatic` с начальным значением 0

Добавьте в класс `TestClass` публичный статический метод `PI`, возвращающий значение `Math.PI`

Добавьте в класс `TestClass` публичный статический метод `ceil`, возвращающий число округленное в большую сторону

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение работать с классами в TypeScript.
- Умение добавлять публичные поля класса.
- Умение создавать и использовать статические поля класса.
- Умение создавать публичные статические методы.

- Умение использовать метод `Math.ceil` для округления числа.

Ссылка на репозиторий

<https://github.com/tensor-school/task-static-fields>

Задание 17

Задание по теме «Асинхронность в JS»

Описание

Необходимо выполнить задания по работе с асинхронным кодом и `Promise`. Требуется реализовать функцию, которая возвращает `Promise` и завершается через указанное количество миллисекунд, ускорить выполнение одной функции, а также исправить другую функцию, чтобы она корректно обрабатывала завершённый с ошибкой `Promise` и возвращала текст ошибки.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/Async.ts` выполните следующие задания:

Создайте функцию `mock`, которая принимает на вход аргумент `number` (количество миллисекунд) и возвращает `Promise`, который завершится через заданное количество миллисекунд со значением, переданным в аргумент.

Перепишите функцию `getData` так, чтобы она выполнялась быстрее.

Исправьте функцию `catchException` так, чтобы блок `try/catch` обрабатывал завершённый с ошибкой `Promise` и возвращал текст ошибки.

Проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте `pull request` с решением.

Что проверяем

- Умение работать с `Promise` в TypeScript.
- Умение создавать асинхронные операции с задержкой выполнения.
- Понимание последовательного и параллельного выполнения асинхронных операций.
- Умение оптимизировать выполнение нескольких независимых асинхронных задач.
- Умение обрабатывать ошибки в асинхронном коде.
- Понимание особенностей использования `try/catch` при работе с `Promise`.
- Умение возвращать результат асинхронной операции.

Ссылка на репозиторий

<https://github.com/tensor-school/task-async>

Задание 18

Задание по теме «Рендеринг JSX»

Описание

Необходимо создать компонент, который отображает название выбранного источника данных и последнее сообщение, полученное от него. Компонент должен принимать пропс, подписываться на изменения источника, обновлять отображаемое сообщение при получении события и корректно отписываться через `unsub` при смене источника. После переключения на новый источник до получения первого сообщения необходимо выводить значение `-1`.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/Effects.tsx` вам нужно будет создать компонент, который будет отображать название выбранного источника данных и последнее сообщение, которое пришло от источника. Также,

этот компонент должен уметь реагировать на изменение источника:

Постройте компонент, выводящий информацию об источнике. Он будет получать проп `sourceId`: `string`.

Информация должна выводиться в следующем виде:

`<Название>`: `<последнее_сообщение>`

После смены источника считается, что сообщений от него ещё не было. В этом случае нужно выводить

`<Название>`: -1

Подпишитесь на изменение статуса пользователя с помощью метода `subscribe(id, callback)`. В обработчик будет приходить новое сообщение, который нужно отрисовать.

При изменении пользователя нужно отписываться от изменения его статуса с помощью `unsubscribe(id, callback)`.

Функции `subscribe(id, callback)` и `unsubscribe(id, callback)` нужно импортировать из `src/resources/API.ts`

Используйте команду `npm run start`, чтобы в вашем браузере открылась страница с вашими компонентами, а также парой кнопок, которые помогут вам протестировать поведение. Кнопка `Change source` переключается между двумя источниками, а кнопка `Fire event` стреляет событием для ВСЕХ источников.

Так же, проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение создавать React-компоненты с пропсами.
- Умение работать с побочными эффектами в React.
- Умение подписываться на внешние события.
- Умение корректно выполнять очистку эффекта и отписываться от событий.
- Понимание зависимости эффекта от изменяемого источника данных.
- Умение обновлять состояние компонента при получении нового сообщения.
- Умение обрабатывать смену источника и сбрасывать отображаемое значение.
- Умение импортировать и использовать функции из внешнего модуля.
- Умение проверять работу компонента через интерфейс приложения.

Ссылка на репозиторий

<https://github.com/tensor-school/task-react-render-and-effects>

Задание 19

Задание по теме «Списки»

Описание

Необходимо создать компонент со списком редактируемых элементов. Компонент должен принимать два пропа: направление сортировки и массив, где каждый элемент содержит идентификатор и имя. Необходимо отображать имена элементов, позволять редактировать имя по клику, сохранять изменения и отменять их. При изменении направления сортировки по идентификатору состояние редактируемого элемента и введённые пользователем данные не должны теряться.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/Keys.tsx` вам нужно будет создать список из редактируемых элементов:

На вход в компонент `Keys` будет приходить 2 пропса: `sorting: 'ASC' | 'DESC'` и `initialData: IItem[]`, где `IItem` это:

Нужно вывести поле `name` каждого элемента.

При клике на имя оно должно превращаться в `<input>`. С помощью этого поля ввода имя можно редактировать, сохранение должно происходить по нажатию `Enter`, отмена изменений по нажатию `Escape`.

Вверху страницы есть кнопка `Change sorting`. При клике на неё меняется направление сортировки по полю `id`. Если эта кнопка нажимается во время редактирования, то текущее состояние должно сохраняться. Т.е. редактируемый элемент должен остаться редактируемым, если пользователь что-то ввёл – его изменения не должны исчезнуть.

Используйте команду `npm run start`, чтобы в вашем браузере открылась страница, на которой можно проверить работу вашего компонента.

Так же, проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение создавать React-компоненты с типизированными пропсами.
- Умение работать со списками элементов в React.
- Понимание назначения и правильного использования ключей `key`.
- Умение хранить и изменять локальное состояние компонента.
- Умение переключать элемент между режимами просмотра и редактирования.
- Умение обрабатывать события клавиатуры `Enter` и `Escape`.
- Умение сортировать данные по полю `id` в прямом и обратном порядке.
- Понимание того, как сохранить состояние элемента при изменении порядка элементов в списке.
- Умение проверять работу компонента через интерфейс приложения.

Ссылка на репозиторий

<https://github.com/tensor-school/task-keys>

Задание 20

Задание по теме «Контекст»

Описание

Необходимо реализовать передачу темы приложения через React Context вместо последовательной передачи пропсов между компонентами. Нужно создать компонент, который принимает тему "светлую" или "тёмную" и передаёт её дочерним компонентам через контекст, а также хук для получения текущей темы. После необходимо заменить передачу темы через пропсы на использование созданного контекста.

Текст задания

Установите зависимости:

```
npm install
```

В файле `src/App.tsx` в компоненте `<App>` есть кнопка, переключающая тему и компонент `<List>`. Компонент `<List>` принимает в себя проп `theme: 'light' | 'dark'`. Список передаёт тему в каждый из компонентов `<ListItem>`, которые используют этот проп для выбора класса.

В файле `src/Context.tsx` реализуйте следующее:

Компонент `ThemeProvider`, который будет принимать проп `theme: 'light' | 'dark'` и распространять его вниз через контекст.

Хук `useTheme`, который будет получать тему из контекста.

Затем, вернитесь в файл `src/App.tsx` и замените передачу пропсов на использование контекста.

Используйте команду `npm run start`, чтобы в вашем браузере открылась страница, на которой можно проверить работу вашего компонента.

Так же, проверить себя можно запустив команду `npm run test`.

После выполнения задания создайте pull request с решением.

Что проверяем

- Умение создавать и использовать React Context.
- Умение создавать компонент-провайдер для передачи данных дочерним компонентам.
- Умение создавать пользовательские хуки.

- Умение получать данные из контекста внутри компонентов.
- Понимание проблемы последовательной передачи пропсов между компонентами.
- Умение заменять передачу данных через пропсы на использование контекста.
- Умение работать с типизированными значениями `'light' | 'dark'`.
- Умение проверять работу компонента через интерфейс приложения.

Ссылка на репозиторий

<https://github.com/tensor-school/task-context>